Università di Genova

UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

Second Assignment

Manipulator Geometry and Direct Kinematics

Author:

Beaini Charbel
Gjinaj Endri
Topulli Joel

Student ID:

S8027823
S8663382
S5418701

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Simone Borelli
Luca Tarasi

December 17, 2025

Contents

Mathematical expression	Definition	MATLAB expression
$< w >$	World Coordinate Frame	w
${}^{a}_{b}R$	Rotation matrix of frame $< b >$ with respect to frame $< a >$	aRb
${}^{a}_{b}T$	Transformation matrix of frame $< b >$ with respect to frame $< a >$	aTb

Table 1: Nomenclature Table

1 Assignment description

The second assignment of Modelling and Control of Manipulators focuses on manipulators' geometry and direct kinematics.

- Download the .zip file from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling the template classes called *geometricModel* and *kinematicModel*
- Write a report motivating your answers, following the predefined format on this document.

1.1 Exercise 1

Given the following CAD model of an industrial 7 DoF manipulator:

Q1.1 Define all the model transformation matrices, by filling the structures in the *BuildTree()* function. Be careful to define the z-axis coinciding with the joint rotation axis, such that the positive rotation is the same as showed in the CAD model you received; and the x-axis, when possible, pointing to the next link. Draw on the CAD model the reference frames for each link and insert it into the report.

Q1.2 Implement the method of *geometricModel* called *updateDirectGeometry()* which computes the model transformation matrices as a function of the joint position q. Explain the method used and comment on the results obtained.

Q1.3 Implement the method of *geometricModel* called *getTransformWrtBase()* which computes the transformation matrix from the base to a given frame. Using this method, compute the following transformation matrices: ${}^{b}_{e}T$, ${}^{6}_{2}T$. Explain the method used and comment on the results obtained.

Q1.4 Run the MATLAB section Q1.4 in the given script *main.m*, which computes the configurations of the manipulators moving from an initial joint position $q_i = [\frac{\pi}{4}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.15, \frac{\pi}{4}]^\top$ to $q_f = [\frac{5\pi}{12}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.18, \frac{\pi}{5}]^\top$. The script generates plots using methods from the given class *plotManipulators*, which must not be modified. Check that it behaves reasonably and show the obtained plots in the report.

Q1.5 Implement the method of *kinematicModel* called *getJacobianOfLinkWrtBase()* which takes as argument the link number, and returns the Jacobian of the corresponding link with respect to the base. Compute the Jacobian of **link 6** with respect to base for the final configuration q_f given in Q1.4. Explain the method used and comment on the result obtained.

Q1.6 Implement the method of *kinematicModel* called *updateJacobian()* which updates the Jacobian matrix of the manipulator for the end-effector. Compute the Jacobian for the final configuration q_f given in Q1.4. Explain the method used and comment on the results obtained.

Q1.7 Given the following joint position, $q = [0.7, -0.1, 1, -1, 0, 0.03, 1.3]^\top$ expressed in radians and meters, and the following joint velocities $\dot{q} = [0.9, 0.1, -0.2, 0.3, -0.8, 0.5, 0]^\top$ expressed in rad/s and m/s, compute the velocities of the end effector with respect to the base projected on the end effector frame, ${}^{e}v_{e/b}$ and ${}^{e}\omega_{e/b}$, using **forward kinematics**.

Remark: All the methods must be implemented for a generic serial manipulator. For instance, joint types, and the number of joints should be parameters.

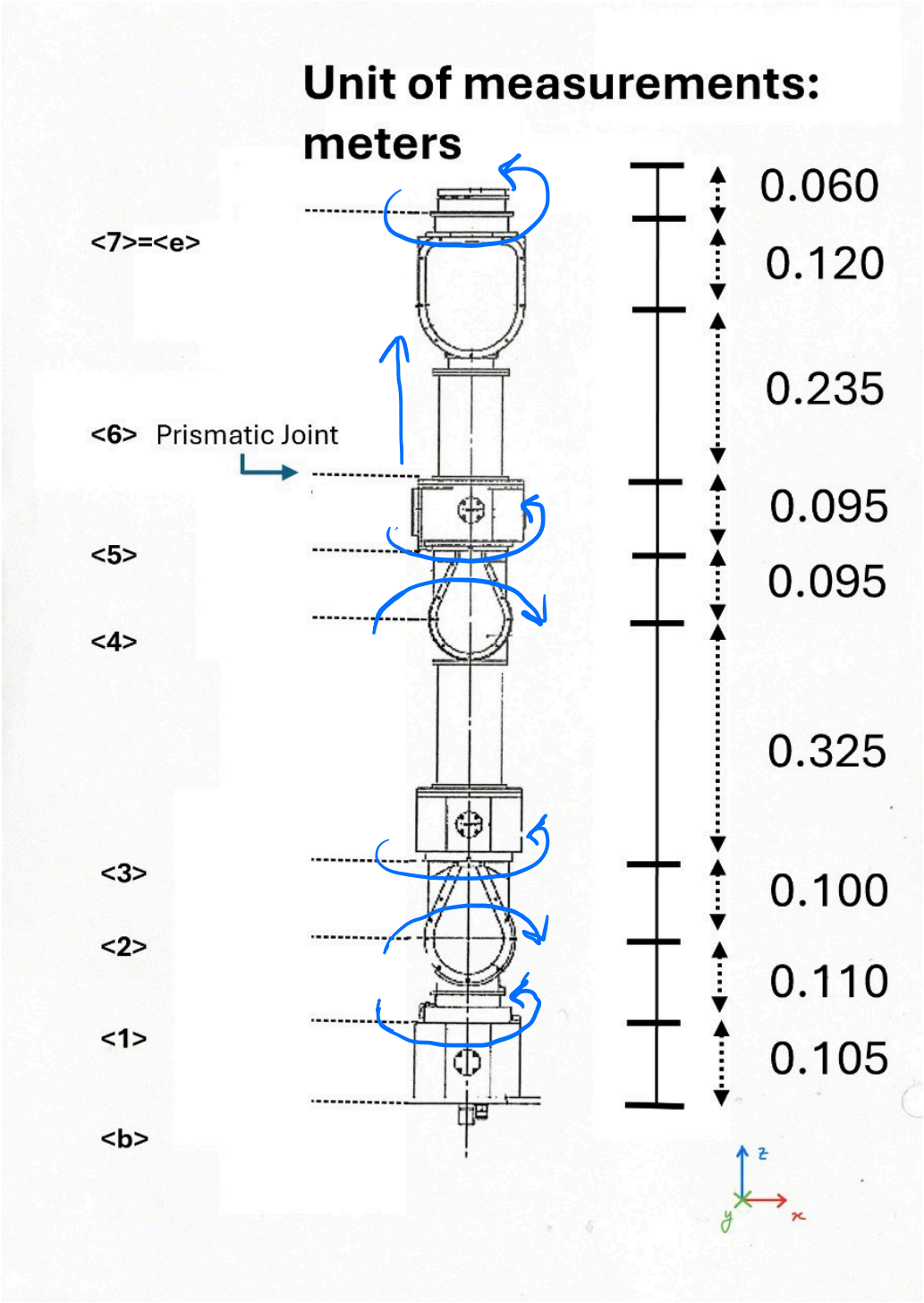

Figure 1: CAD model of the robot. The directions of motion of each joint are indicated by the blue arrows.

2 Exercise 1

2.1 Q1.1

In this exercise, the reference frames associated with the seven joints of the given manipulator are defined, together with the corresponding constant homogeneous transformation matrices used in the kinematic model. All rotations are constructed using the standard elementary rotation matrices about the coordinate axes, following the right-hand rule for the sign convention. The general form of a homogeneous transformation between two consecutive frames is:

$$ {}^{i-1}_{i}T = \begin{bmatrix} {}^{i-1}_{i}R & {}^{i-1}_{i}P \\ \mathbf{0}_{1\times 3} & 1 \end{bmatrix}, \tag{1} $$

where ${}^{i-1}_{i}R$ is the rotation matrix and ${}^{i-1}_{i}P$ is the position of the origin of frame $\{i\}$ expressed in frame $\{i-1\}$.

The assignment requires defining all model transformation matrices inside `BuildTree()`, ensuring that:

- the z-axis of each frame coincides with the corresponding joint rotation axis, with the positive direction matching the arrows indicated in Figure 1;
- the x-axis points toward the next link whenever possible;
- the y-axis is determined by the right-hand rule applied to the chosen x- and z-axes.

These conventions ensure consistency across the full kinematic chain and allow the joint motions to be correctly represented in subsequent computations.

The frame locations and orientations were first assigned directly on the CAD model. The resulting structure was then validated using the same plotting tools implemented in the previous assignment. In particular, the function `PlotRefFrames` (built upon our custom `plotFrame` and `homog` utilities) was reused to visualize all reference frames in 3D. This produced the frame tree shown in Figure 2, confirming that the hand-defined transformations were consistent.

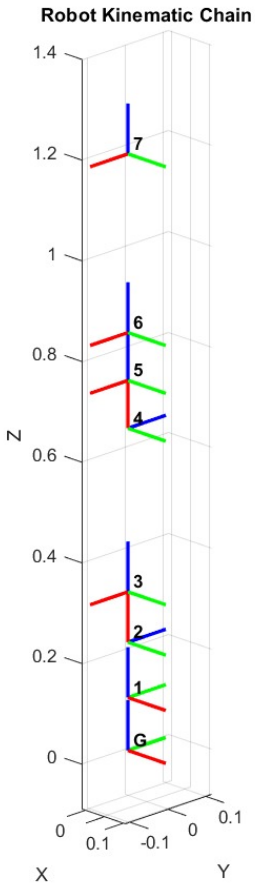

Figure 2: Reference frames assigned to each link using the CAD geometry and the standard axis conventions. The unit of measurement is meters.

Using the reference configuration $q_0 = 0$, the constant transformations implemented in `BuildTree()` are:

$$ {}^{0}_{1}T(0) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.105 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{1}_{2}T(0) = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0.110 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{2}_{3}T(0) = \begin{bmatrix} 0 & 0 & 1 & 0.100 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{3}_{4}T(0) = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0.325 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{4}_{5}T(0) = \begin{bmatrix} 0 & 0 & 1 & 0.095 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad {}^{5}_{6}T(0) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.095 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{6}_{7}T(0) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.355 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

A brief interpretation of each transform is given below, in line with the comments in the `BuildTree()` code:

- ${}^{0}_{1}T(0)$: There is no rotation between frame $\{0\}$ and frame $\{1\}$, so the rotational part is the identity matrix. The origin of frame $\{1\}$ is located 0.105 m from frame $\{0\}$ along the z_0 direction, which appears in the fourth column.
- ${}^{1}_{2}T(0)$: This transform corresponds to a fixed reorientation equivalent to a rotation $\mathrm{R}_x(-90^\circ)\mathrm{R}_z(-90^\circ)$, followed by a translation of 0.110 m along the local z-axis of frame $\{1\}$. The z-axis of frame $\{2\}$ is aligned with the second joint axis, as required.
- ${}^{2}_{3}T(0)$: The rotation part is equivalent to a rotation $\mathrm{R}_y(90^\circ)$, which reorients the z-axis while keeping the y-axis unchanged. The translation of 0.100 m appears along the local x-axis of frame $\{2\}$.
- ${}^{3}_{4}T(0)$: This transform represents a rotation $\mathrm{R}_y(-90^\circ)$ and a translation of 0.325 m along the z-axis of frame $\{3\}$, corresponding to the main length of this link.
- ${}^{4}_{5}T(0)$: Again, the rotation part is equivalent to $\mathrm{R}_y(90^\circ)$, with a translation of 0.095 m along the local x-axis of frame $\{4\}$.
- ${}^{5}_{6}T(0)$: There is no relative rotation between frames $\{5\}$ and $\{6\}$; the orientation is preserved, therefore the only effect is a translation of 0.095 m along the common z-axis.
- ${}^{6}_{7}T(0)$: The last transform is a pure translation of 0.355 m along the z-axis, from frame $\{6\}$ to the end-effector frame $\{7\}$, with identical orientation.

These successive transformations specify the geometry of the manipulator in its home position and serve as the foundation for all forward kinematics computations in the following questions. Their correctness has been verified visually by plotting the functions developed earlier (Figure 2), ensuring that the implemented `BuildTree()` structure represents the actual CAD model (Figure 1) accurately.

2.2 Q1.2

This section extends the geometric model introduced in Section 2.1 by incorporating the joint variables into the link transformations. The matrices ${}^{i-1}_{i}T(0)$ describe the geometry of the manipulator in its home position. However, to model the robot in motion, they must be defined as a function of the joint configuration q. Henceforth the method `updateDirectGeometry()` implements this update by generating a new set of matrices ${}^{i-1}_{i}T(q_i)$, one for each joint.

The update relies on the frame convention established in 2.1, where each joint axis is aligned with the z-axis of its corresponding frame. This choice leads to a compact representation of joint actuation. The updated transformation is computed as

$$ {}^{i-1}_{i}T(q_i) = {}^{i-1}_{i}T(0)\, T_{\text{joint},i}(q_i), \tag{2} $$

where the joint contribution $T_{\text{joint},i}(q_i)$ is always a pure rotation or translation about the local z-axis.

The form of $T_{\text{joint},i}(q_i)$ depends on the joint type. For a revolute joint, the joint contribution is a rotation about the local $z - axis$:

$$ T_{\text{joint},i}(q_i) = \begin{bmatrix} \cos(q_i) & -\sin(q_i) & 0 & 0 \\ \sin(q_i) & \cos(q_i) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{3} $$

For a prismatic joint, the contribution is a translation along the same axis:

$$ T_{\text{joint},i}(q_i) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & q_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{4} $$

For each joint i, the static transform ${}^{i-1}_{i}T(0)$ is retrieved, the joint-specific matrix $T_{\text{joint},i}(q_i)$ is constructed according to the joint type, and the updated transform is obtained through (2). This produces a complete set of relative transformations that reflect both the fixed geometry and the current joint configuration.

To verify the behavior of the method, the joint configuration defined in Section 2.4 was evaluated:

$$ q_i = \left[\tfrac{\pi}{4},\ -\tfrac{\pi}{4},\ 0,\ -\tfrac{\pi}{4},\ 0,\ 0.15,\ \tfrac{\pi}{4}\right]^\top \tag{5} $$

The updated matrices ${}^{i-1}_{i}T(q_i)$ produced by the method are:

$$ {}^{0}_{1}T(q) = \begin{bmatrix} 0.7071 & -0.7071 & 0 & 0 \\ 0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.1050 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} \quad {}^{1}_{2}T(q) = \begin{bmatrix} -0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1.0000 & 0 \\ 0.7071 & 0.7071 & 0 & 0.1100 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

$$ {}^{2}_{3}T(q) = \begin{bmatrix} 0 & 0 & 1.0000 & 0.1000 \\ 0 & 1.0000 & 0 & 0 \\ -1.0000 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} \quad {}^{3}_{4}T(q) = \begin{bmatrix} 0 & 0 & -1.0000 & 0 \\ -0.7071 & 0.7071 & 0 & 0 \\ 0.7071 & 0.7071 & 0 & 0.3250 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

$$ {}^{4}_{5}T(q) = \begin{bmatrix} 0 & 0 & 1.0000 & 0.0950 \\ 0 & 1.0000 & 0 & 0 \\ -1.0000 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} \quad {}^{5}_{6}T(q) = \begin{bmatrix} 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.2450 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

$$ {}^{6}_{7}T(q) = \begin{bmatrix} 0.7071 & -0.7071 & 0 & 0 \\ 0.7071 & 0.7071 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.3550 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

The structure of the matrices confirms the expected behavior for each joint. When a joint angle is non-zero, the rotation about the corresponding $z - axis$ appears in the upper-left 2×2 block, while the translation remains consistent with the link dimensions defined in 2.1. Joints for which $q_i = 0$ (joints 3 and 5) preserve their reference configuration transforms exactly. The prismatic joint (joint 6) correctly increases only the translation along the z-axis, matching the imposed displacement. A more detailed interpretation of the effect of this configuration on the full kinematic chain will be discussed in Section 2.4.

2.3 Q1.3

The method `getTransformWrtBase()` provides a general tool to compute the transformation between any two frames of the manipulator, using the updated relative transformations obtained in Section 2.2. After `updateDirectGeometry()` has populated the matrices ${}^{i-1}_{i}T(q_i)$ inside `iTj`, the transformation between two arbitrary frames $\{s\}$ and $\{k\}$ is obtained by traversing the kinematic chain.

If the start index satisfies $s < k$, the method computes the forward product of all relative transformations between frames $\{s\}$ and $\{k\}$:

$$ {}^{s}T_k = {}^{s}T_{s+1}\ {}^{s+1}T_{s+2}\ \cdots\ {}^{k-1}T_k. \tag{6} $$

If $s = k$, the resulting transform is the identity matrix. When $s > k$, the method first builds the forward product from frame $\{k\}$ to frame $\{s\}$,

$$ {}^{k}T_s = {}^{k}T_{k+1}\ {}^{k+1}T_{k+2}\ \cdots\ {}^{s-1}T_s, \tag{7} $$

and then inverts the resulting homogeneous transformation. The analytical inverse is

$$ T^{-1} = \begin{bmatrix} R^\top & -R^\top p \\ 0 & 1 \end{bmatrix}. \tag{8} $$

This formulation allows the computation of transformations between any pair of frames, not only from the base. Below, the method is applied to the joint configuration introduced in equation (5).

Base to End–Effector Transform,

$$ {}^{b}T_e = \begin{bmatrix} 0.5000 & -0.5000 & -0.7071 & -0.7039 \\ -0.5000 & 0.5000 & -0.7071 & -0.7039 \\ 0.7071 & 0.7071 & 0.0000 & 0.5155 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}. $$

The last column gives the end-effector position, approximately 0.52 m above the base and with equal negative offsets in x and y. The rotation block reflects the chain of joint rotations: the end-effector z-axis lies in the horizontal plane, while the x- and y-axes appear rotated by about 45° relative to the base frame, which is consistent with the active joints in this configuration.

Transform from Frame $\{6\}$ to Frame $\{2\}$

$$ {}^{6}T_2 = \begin{bmatrix} 0 & 0 & -1.0000 & 0 \\ 0.7071 & 0.7071 & 0 & -0.3005 \\ 0.7071 & -0.7071 & 0 & -0.6405 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}. $$

This matrix expresses frame $\{2\}$ relative to frame $\{6\}$. The rotational part shows that the z-axis of $\{2\}$ aligns with the negative x-direction of $\{6\}$, while the other axes form a 45° rotation in the orthogonal plane. The translation components arise from the accumulated offsets between frames $\{2\}$ and $\{6\}$ along the kinematic chain.

Although the method computes ${}^{6}T_2$ directly, the original version of the function did not allow specifying both a start and an end frame. We extended the interface so that the user can request the transformation between any two frames, which simplifies the computation of intermediate transforms such as ${}^{6}T_2$.

An alternative to modifying the method is to compute transformations between arbitrary frames using only transforms expressed with respect to the base. In general, if ${}^{0}T_a$ and ${}^{0}T_b$ denote the transformations of frames $\{a\}$ and $\{b\}$ with respect to the base frame, then the transform from $\{a\}$ to $\{b\}$ is given by

$$ {}^{a}T_b = \left({}^{0}T_a\right)^{-1}\ {}^{0}T_b. \tag{9} $$

Applying (9) to the specific case yields the two transforms

$$ {}^{0}T_{6} \quad \text{and} \quad {}^{0}T_{2}, $$

and their combination gives

$$ {}^{6}T_{2} = \left({}^{0}T_{6}\right)^{-1} {}^{0}T_{2}, \tag{10} $$

which follows from the chain rule for homogeneous transformations. The result obtained from (10) matches the value produced by `getTransformWrtBase()`, confirming the accuracy of the implementation and the consistency of both forward and reverse traversal of the kinematic chain.

2.4 Q1.4

In this section, the manipulator is moved from the initial joint configuration defined in equation (5), to the final configuration:

$$ q_f = \left[\tfrac{5\pi}{12},\ -\tfrac{\pi}{4},\ 0,\ -\tfrac{\pi}{4},\ 0,\ 0.18,\ \tfrac{\pi}{5}\right]^{\top}. \tag{11} $$

The trajectory in joint space is obtained by linearly interpolating each joint angle between q_i and q_f over a time interval of 10 seconds using 100 samples. For each intermediate configuration q, the method `updateDirectGeometry()` updates the relative transforms ${}_{i}^{i-1}T(q_i)$, and `getTransformWrtBase()` is then used to compute the transforms ${}^{b}T_{k}$ from the base frame $\{b\}$ to every frame $\{k\}$ along the chain.

The class `plotManipulators` takes these base-relative transforms and produces two types of plots:

- a 3D plot of the motion of the manipulator over time, where at each iteration the positions of the base and all intermediate frames are connected by a line;
- a final configuration plot, where only the last pose of the manipulator is shown.

In both cases, the values defined at:

$$ {}^{b}r_{k} = {}^{b}T_{k}(1{:}3, 4), \quad k = 0, \ldots, 7, $$

are used to construct the polyline that approximates the robot structure in space.

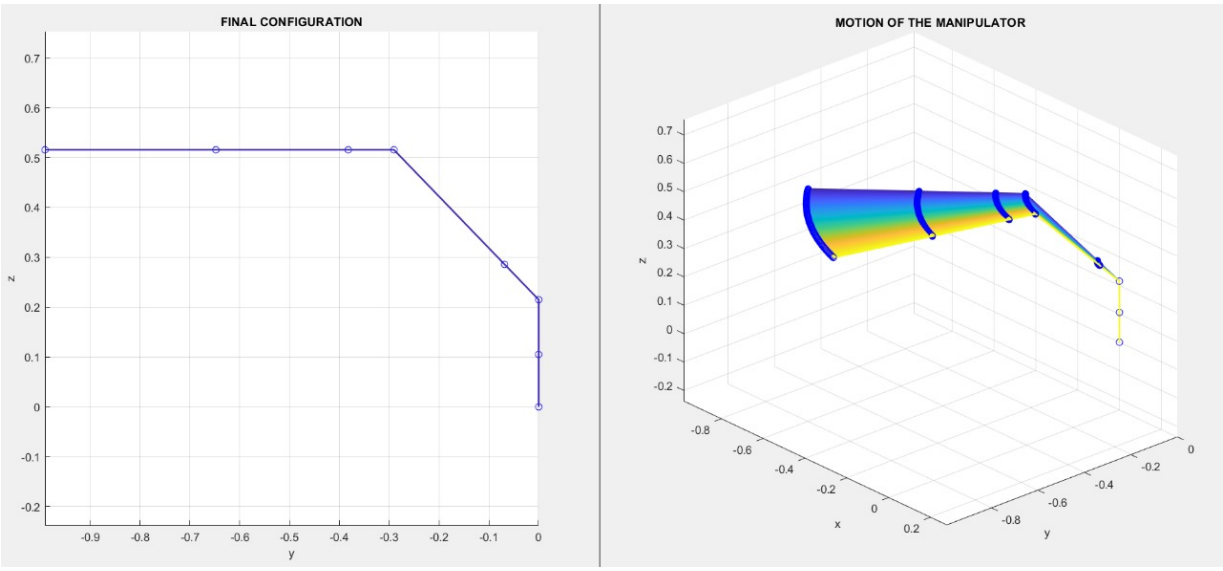

Figure 3: On the left, the final configuration of the manipulator in the (y, z)-plane. On the right, the full 3D motion of the manipulator during the joint-space interpolation from q_i to q_f.

The right-hand plot in Figure 3 shows the motion of the manipulator in 3D. At each time step, the polyline connecting the frames $\{0\}, \{1\}, \ldots, \{7\}$ is drawn, and a different color is assigned according to the current time index. As a result, the sequence of lines forms a smooth plane that represents how the links sweep through space as the joints move from q_i to q_f.

Joint 1 rotates about the base z-axis from $\pi/4$ to $5\pi/12$, causing the whole arm to swing in the horizontal plane, while joint 2 remains constant. The prismatic joint (joint 6) extends from 0.15 m to 0.18 m, which produces a small radial displacement of the end-effector outward from the base.

The left-hand plot in Figure 3 shows only the final configuration obtained at q_f, viewed in such a way that the manipulator is projected onto the (y, z)-plane. The base frame is located at the origin, and the successive joint positions form a continuous chain that rises in z and extends mainly in the negative y direction.

These results confirm that the combination of `updateDirectGeometry()`, `getTransformWrtBase()`, and `plotManipulators` provides a correct and visually consistent representation of the manipulator motion between the two given joint configurations.

2.5 Q1.5

In this section, the geometric Jacobian of link 6 with respect to the base frame is computed and evaluated at the final joint configuration q_f given in (11). The goal is to obtain a matrix $J_6(q) \in \mathbb{R}^{6\times 7}$ that maps joint velocities $\dot{q}$ to the instantaneous spatial twist (**linear and angular velocity**) of the origin of frame $\{6\}$, expressed in the base frame $\{b\}$.

For a serial manipulator, the instantaneous motion of a frame can be described by the spatial twist:

$$ {}^{b}V_6 \;=\; \begin{bmatrix} {}^{b}v_6 \\ {}^{b}\omega_6 \end{bmatrix} \in \mathbb{R}^6, \tag{12} $$

where ${}^{b}v_6$ is the linear velocity of the origin of frame $\{6\}$ and ${}^{b}\omega_6$ is its angular velocity, both expressed in the base frame. The geometric Jacobian $J_6(q)$ is defined by the linear mapping:

$$ {}^{b}V_6 \;=\; J_6(q)\,\dot{q}, \tag{13} $$

with $n = 7$ joints in the chain. A useful way to interpret (13) is to expand it column-wise:

$$ {}^{b}V_6 \;=\; \sum_{j=1}^{7} J_6(:, j)\,\dot{q}_j. \tag{14} $$

Thus, the j-th column $J_6(:, j)$ represents the instantaneous twist generated at link 6 when *only* joint j moves with unit speed ($\dot{q}_j = 1$) while all other joints are locked ($\dot{q}_k = 0,\ k \neq j$). This superposition principle follows from the first-order properties of rigid-body kinematics.

The Jacobian is built using the standard geometric formulation based on joint axes. Let ${}^{b}T_j(q)$ be the homogeneous transformation from the base frame to frame $\{j\}$. From this transform we extract:

$$ {}^{b}p_j = {}^{b}T_j(1{:}3, 4), \qquad {}^{b}z_j = {}^{b}T_j(1{:}3, 3), \tag{15} $$

where ${}^{b}p_j$ is the position of the origin of frame $\{j\}$ and ${}^{b}z_j$ is the z-axis of frame $\{j\}$, i.e., the joint axis direction expressed in the base frame under the adopted frame assignment. The position of the target frame is:

$$ {}^{b}p_6 = {}^{b}T_6(1{:}3, 4). \tag{16} $$

For each joint $j \leq 6$, define the distance vector from the joint axis point to the target point,

$$ {}^{b}r_{6/j} = {}^{b}p_6 - {}^{b}p_j. \tag{17} $$

Then the j-th Jacobian column is computed as:

- Revolute joint ($j \in \{1, \ldots, 5\}$ and 7 in this manipulator):

$$ J_v(:, j) = {}^{b}z_j \times {}^{b}r_{6/j}, \qquad J_\omega(:, j) = {}^{b}z_j. \tag{18} $$

- Prismatic joint (joint 6 in this manipulator):

$$ J_v(:, 6) = {}^{b}z_6, \qquad J_\omega(:, 6) = 0. \tag{19} $$

Finally, the successive joints of the considered link do not affect it: for link 6, joint 7 contributes a zero column, i.e., $J_6(:,7) = 0$.

Evaluating the above construction at $q = q_f$ yields the Jacobian of link 6 with respect to the base:

$$J_6(q_f) = \begin{bmatrix} 0.6477 & 0.0778 & 0.2527 & 0.0000 & -0.0000 & -0.2588 & 0 \\ -0.1735 & 0.2903 & -0.0677 & 0.0000 & 0.0000 & -0.9659 & 0 \\ 0.0000 & 0.6705 & 0.0000 & 0.3700 & 0.0000 & 0.0000 & 0 \\ 0.0000 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0.0000 & 0 \\ 0.0000 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0.0000 & 0 \\ 1.0000 & 0.0000 & 0.7071 & 0.0000 & 0.0000 & 0.0000 & 0 \end{bmatrix}. \tag{20}$$

The structure of (20) is physically meaningful when interpreted through (14). In what follows, each column is read as the twist produced at link 6 by a unit joint rate.

- Column 1 (joint 1, revolute): the angular part is $[0 \;\; 0 \;\; 1]^\top$, indicating that joint 1 rotates about the base z-axis. The linear part lies in the (x, y)-plane ($[0.6477 \;\; -0.1735 \;\; 0]^\top$), consistent with a tangential velocity generated by rotating the entire arm around the vertical axis. The absence of a z component in the linear part is coherent with an instantaneous rotation about z_b.
- Column 2 (joint 2, revolute): the angular part is approximately a unit vector in the base (x, y)-plane ($[-0.9659 \;\; 0.2588 \;\; 0]^\top$), showing that the joint axis is tilted with respect to the base axes at q_f. The linear part includes a z component (0.6705), which produces significant vertical motion of frame 6.
- Column 3 (joint 3, revolute): the angular part $[-0.1830 \;\; -0.6830 \;\; 0.7071]^\top$ indicates a general 3D axis direction, hence a spatial rotation not aligned with any base axis. The corresponding linear part is produced by the moment arm via the cross product; its magnitude reflects both the distance from the axis to the origin of frame 6 and the current posture.
- Column 4 (joint 4, revolute): the angular part is identical to column 2 ($[-0.9659 \;\; 0.2588 \;\; 0]^\top$), meaning that joints 2 and 4 have parallel axes at this configuration. The linear part differs because joint 4 is located further along the chain.
- Column 5 (joint 5, revolute): the angular part $[-0.2588 \;\; -0.9659 \;\; 0]^\top$ is again a unit axis direction lying in the (x, y)-plane. The linear part is zero ($[0 \;\; 0 \;\; 0]^\top$). This occurs when the origin of frame 6 lies on (or very close to) the axis of joint 5, so that ${}^b r_{6/5}$ is effectively zero or collinear with ${}^b z_5$, causing the cross product to effectively vanish. In this case, joint 5 changes the orientation of frame 6 without translating its origin.
- Column 6 (joint 6, prismatic): the angular part is exactly zero, since a prismatic joint cannot induce rotation. The linear part equals the translation axis expressed in the base frame, $[-0.2588 \;\; -0.9659 \;\; 0]^\top$. The motion lies entirely in the (x, y)-plane, indicating that the prismatic joint is oriented horizontally in the base frame at this configuration.
- Column 7 (joint 7, revolute): the column is identically zero because joint 7 is downstream of link 6. Rotating joint 7 changes the pose of link 7 (the end-effector) but cannot affect the pose of link 6 in a serial chain.

Overall, the computed Jacobian in (20) matches the expected kinematic behavior of a serial 7-DoF manipulator. The results confirm that the implemented method `getJacobianOfLinkWrtBase()` correctly captures the differential relationship between joint rates and the spatial twist of link 6 with respect to the base frame.

2.6 Q1.6

In this section, the method `updateJacobian()` is implemented in order to update the manipulator's Jacobian for the end-effector. In the adopted convention, the end-effector corresponds to the last frame of the chain (frame $\{7\}$). The objective is to compute the geometric Jacobian $J_e(q) \in \mathbb{R}^{6\times 7}$ that maps joint velocities $\dot{q}$ to the instantaneous twist of the end-effector expressed in the base frame $\{b\}$:

$${}^b V_e \;=\; \begin{bmatrix} {}^b v_e \\ {}^b \omega_e \end{bmatrix} \;=\; J_e(q)\,\dot{q}. \tag{21}$$

The method `updateJacobian()` updates the internal Jacobian stored in the kinematic model object by calling the already implemented routine `getJacobianOfLinkWrtBase()` with the index of the last link. Since the geometric model has already been updated to the desired configuration through `updateDirectGeometry()`, the Jacobian columns are computed using the same axis-based formulation described in section 2.5. For each

joint j the joint axis direction ${}^{b}z_j$ and joint origin ${}^{b}p_j$ are extracted from ${}^{b}T_j(q)$, while the end-effector position ${}^{b}p_e$ is extracted from ${}^{b}T_7(q)$.

Evaluating the end-effector Jacobian at the final configuration q_f yields:

$$J_{ee}(q_f) = \begin{bmatrix} 0.9906 & 0.0778 & 0.4952 & 0.0000 & -0.0000 & -0.2588 & 0.0000 \\ -0.2654 & 0.2903 & -0.1327 & 0.0000 & 0.0000 & -0.9659 & 0.0000 \\ 0.0000 & 1.0255 & 0.0000 & 0.7250 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0.0000 & -0.2588 \\ 0.0000 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0.0000 & -0.9659 \\ 1.0000 & 0.0000 & 0.7071 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}. \tag{22}$$

Compared with the Jacobian of link 6 computed in 2.5, two differences are particularly meaningful:

- Nonzero contribution of joint 7: for link 6 the last column was remains zero because joint 7 is downstream of link 6. In contrast, for the end-effector's Jacobian the 7-th column is nonzero in the angular part, equal to the end-effector's joint axis direction ${}^{b}z_7$. This is consistent with the fact that joint 7 directly rotates the tool frame. Moreover, the linear part of column 7 is zero, indicating rightfully that the origin of the end-effector frame lies on the axis of joint 7, so that this joint primarily affects orientation rather than translation at the selected point.
- Change in the linear block for upstream joints: the first four columns of the linear part differ from those of J_6 because the point whose velocity is being described changes from ${}^{b}p_6$ to ${}^{b}p_e$. For a revolute joint, the linear contribution is ${}^{b}z_j \times ({}^{b}p - {}^{b}p_j)$; therefore, moving the reference point further along the chain modifies the distance from the joint axis to the point of interest, resulting in a different cross-product value. The fifth and sixth column remain unchanged in the linear block for the same reasons mentioned in 2.5. The angular block remains unchanged for the first six columns because $J_\omega(:, j) = {}^{b}z_j$ depends only on the orientation of the joint axes and is independent of the chosen reference point.

Overall, the obtained matrix (22) is coherent with the physical interpretation of a geometric Jacobian. These observations confirm that `updateJacobian()` correctly updates the manipulator's Jacobian for the end-effector at the configuration q_f.

2.7 Q1.7

In this section, the end-effector linear and angular velocities with respect to the base are computed for the joint configuration

$$q = \begin{bmatrix} 0.7, & -0.1, & 1, & -1, & 0, & 0.03, & 1.3 \end{bmatrix}^\top, \tag{23}$$

and the corresponding joint velocities

$$\dot{q} = \begin{bmatrix} 0.9, & 0.1, & -0.2, & 0.3, & -0.8, & 0.5, & 0 \end{bmatrix}^\top, \tag{24}$$

expressed in rad/s for revolute joints and m/s for the prismatic joint. The required quantities are the end-effector twist components expressed in the end-effector frame, namely ${}^{e}v_{e/b}$ and ${}^{e}\omega_{e/b}$.

After updating the geometric model to the configuration (23), the end-effector Jacobian $J_e(q)$ is computed using `updateJacobian()`. Since the Jacobian returned by the implementation is a spatial Jacobian expressed in the base frame, the end-effector twist in base coordinates is obtained as:

$${}^{b}V_e = \begin{bmatrix} {}^{b}v_e \\ {}^{b}\omega_e \end{bmatrix} = J_e(q)\,\dot{q}. \tag{25}$$

The problem statement requires the same physical velocity expressed in the end-effector's frame $\{e\}$. Let ${}^{b}T_e(q)$ denote the forward-kinematics transform from base to end-effector and let ${}^{b}R_e \in SO(3)$ be its rotation component. Because both the linear and angular velocities in (25) are associated with the same point (the origin of frame $\{e\}$), changing the coordinate frame amounts to a pure rotation of the vectors:

$${}^{e}v_{e/b} = ({}^{b}R_e)^\top\, {}^{b}v_e, \qquad {}^{e}\omega_{e/b} = ({}^{b}R_e)^\top\, {}^{b}\omega_e. \tag{26}$$

Applying (25) (26) for the data in (23) (24) yields the end-effector velocities expressed in the end-effector frame:

The norms of the linear and angular velocities computed using the Jacobian expressed in different frames coincide:

$$\|\mathbf{v}_b\| = 0.685797, \qquad \|\mathbf{v}_e\| = 0.685797, \qquad \|\boldsymbol{\omega}_b\| = 0.892330, \qquad \|\boldsymbol{\omega}_e\| = 0.892330. \tag{27}$$

Furthermore, the linear and angular velocities obtained directly from forward kinematics and expressed in the end-effector frame are:

$$ {}^{e}v_{\mathrm{EE}} = \begin{bmatrix} 0.2656 \\ -0.3768 \\ 0.5077 \end{bmatrix} \text{ m/s}, \qquad {}^{e}\omega_{\mathrm{EE}} = \begin{bmatrix} 0.5585 \\ 0.4440 \\ -0.5359 \end{bmatrix} \text{ rad/s}. \tag{28} $$

The obtained values in (**??**) represent the instantaneous linear and angular velocity of the end-effector with respect to the base, expressed in the moving frame $\{e\}$. The projection step in (26) is essential, since the Jacobian provides ${}^{b}v_e$ and ${}^{b}\omega_e$ in base coordinates, whereas the requested quantities are expressed in the end-effector's coordinates. Since the transformation between $\{b\}$ and $\{e\}$ is a rigid rotation, the Euclidean norms of the linear and angular velocities are preserved under the projection, while the component values change according to the current end-effector orientation.

A Appendix

In order to double-check the correctness of the analytically derived Jacobians, we also implemented a numerical verification routine, `checkJacobianNumerically()`. This function computes the Jacobian via finite differences and compares it against the analytical expression. Two additional helper functions were introduced to support this procedure. These implementations were not used for deriving results, but solely for validation purposes, providing an independent consistency check of the analytical formulation.

A.1 Jacobian Verification for Link 6

The analytical Jacobian for link 6 is

$$ J^{(6)}_{\text{analytical}} = \begin{bmatrix} 0.6477 & 0.0778 & 0.2527 & 0.0000 & -0.0000 & -0.2588 & 0 \\ -0.1735 & 0.2903 & -0.0677 & 0.0000 & 0.0000 & -0.9659 & 0 \\ 0 & 0.6705 & 0.0000 & 0.3700 & 0.0000 & 0.0000 & 0 \\ 0 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0 & 0 \\ 0 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0 & 0 \\ 1.0000 & 0 & 0.7071 & 0 & 0 & 0 & 0 \end{bmatrix}. $$

The corresponding numerical Jacobian is

$$ J^{(6)}_{\text{numerical}} = \begin{bmatrix} 0.6477 & 0.0778 & 0.2527 & 0.0000 & 0 & -0.2588 & 0 \\ -0.1735 & 0.2903 & -0.0677 & 0.0000 & 0 & -0.9659 & 0 \\ 0 & 0.6705 & 0.0000 & 0.3700 & 0 & 0 & 0 \\ 0 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0 & 0 \\ 0 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0 & 0 \\ 1.0000 & 0 & 0.7071 & 0 & 0 & 0 & 0 \end{bmatrix}. $$

The difference between analytical and numerical Jacobians is on the order of 10^{-6}, confirming numerical consistency.

A.2 Jacobian Verification for the End-Effector (Link 7)

The analytical Jacobian for the end-effector is

$$ J^{(7)}_{\text{analytical}} = \begin{bmatrix} 0.9906 & 0.0778 & 0.4952 & 0.0000 & -0.0000 & -0.2588 & 0 \\ -0.2654 & 0.2903 & -0.1327 & 0.0000 & 0.0000 & -0.9659 & 0 \\ 0 & 1.0255 & 0 & 0.7250 & 0 & 0 & 0 \\ 0 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0 & -0.2588 \\ 0 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0 & -0.9659 \\ 1.0000 & 0 & 0.7071 & 0 & 0 & 0 & 0 \end{bmatrix}. $$

The numerical Jacobian is

$$ J^{(7)}_{\text{numerical}} = \begin{bmatrix} 0.9906 & 0.0778 & 0.4952 & 0.0000 & 0 & -0.2588 & 0 \\ -0.2654 & 0.2903 & -0.1327 & 0.0000 & 0 & -0.9659 & 0 \\ 0 & 1.0255 & 0 & 0.7250 & 0 & 0 & 0 \\ 0 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0 & -0.2588 \\ 0 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0 & -0.9659 \\ 1.0000 & 0 & 0.7071 & 0 & 0 & 0 & 0 \end{bmatrix}. $$

The difference between analytical and numerical Jacobians is again on the order of 10^{-6}.

A.3 Velocity Consistency Checks

The norms of the linear and angular velocities computed using the Jacobian expressed in different frames coincide:

$$\|\mathbf{v}_b\| = 0.685797, \qquad \|\mathbf{v}_e\| = 0.685797,$$
$$\|\boldsymbol{\omega}_b\| = 0.892330, \qquad \|\boldsymbol{\omega}_e\| = 0.892330.$$

Furthermore, the linear and angular velocities obtained directly from forward kinematics and expressed in the end-effector frame are

$$v_{\mathrm{EE}} = \begin{bmatrix} 0.2656 \\ -0.3768 \\ 0.5077 \end{bmatrix}, \qquad \boldsymbol{\omega}_{\mathrm{EE}} = \begin{bmatrix} 0.5585 \\ 0.4440 \\ -0.5359 \end{bmatrix}.$$

The difference between these values and those obtained via the Jacobian projection is on the order of 10^{-6}, confirming full consistency between the analytical Jacobian, the numerical approximation, and the forward kinematics computation.